



Name	Mat. Nr.	Email
Yash Khiste	254751	yash.khiste@st.ovgu.de
Saiful Islam	253574	saiful1.islam@st.ovgu.de
Supritha Rajashekaraiah	244941	supritha.rajashekaraiah@ovgu.de

✓ Preparatory task

✓ Task 1. Describe the sampling schematically and use it to explain the formula for sampling in the time domain.

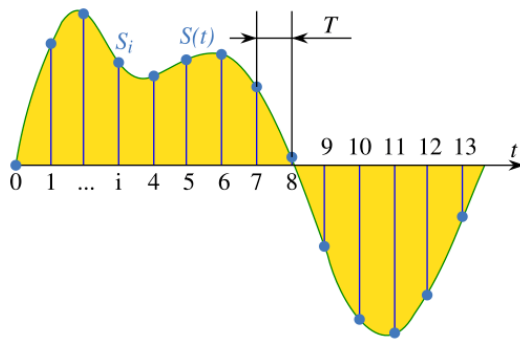
Sampling in the time domain is the process of converting a continuous-time signal into a discrete-time signal by taking periodic samples. This is achieved by multiplying the continuous signal with a train of Dirac delta impulses spaced at regular intervals. Each impulse effectively "captures" the value of the signal at that point in time, which is also the first step in converting an analog signal to a digital signal. According to **Nyquist-Shannon sampling theorem**, to accurately reconstruct the original signal from its samples, the sampling frequency must be at least twice the maximum frequency present in the signal. This minimum rate is called the Nyquist rate.

$$f_s \geq 2f_{\max}$$

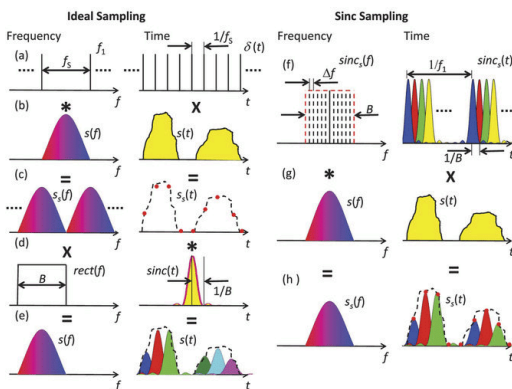
Sampling below this rate leads to aliasing, where high-frequency components fold into lower frequencies and distort the original signal. The ideal reconstruction of the original signal involves passing the sampled signal through a low-pass filter, which removes the spectral replicas and retains only the original frequency content. [\[1\]](#)

Sampling in the time domain is modeled as a multiplication of the continuous-time domain signal $x(t)$ with a periodic train of impulses [\[2\]](#)

$$x_s(t) = x(t) \cdot \sum_{n=-\infty}^{\infty} \delta(t - nT_s)$$



Fig_1: Schematic of sampling a continuous-time signal at uniform intervals. [\[3\]](#)



Fig_2: Sampling of a signal in the frequency and time domain with a Dirac-Delta function (left), corresponding to "ideal" sampling and a sinc-pulse sequence (right). The × and * denote the signs for a multiplication and convolution, respectively. The = marks the corresponding result of the operation. [\[4\]](#)

✓ References

1. ¹Oppenheim, A. V., & Schafer, R. W. (2010). Discrete-Time Signal Processing (3rd ed.). Pearson.
<https://www.pearson.com/store/p/discrete-time-signal-processing/P100000680641>
2. ²A. Wendemuth, E. Andelic, S. Barth, M. Katz, S. Krüger, M. Mamsch, and M. Schafföner, Grundlagen der digitalen Signalverarbeitung: Ein mathematischer Zugang. Berlin, Germany: Springer-Verlag, 2006
3. ³Image source: Wikipedia contributors. "Sampling (signal processing)." Wikipedia, Wikimedia Foundation.
[https://en.wikipedia.org/wiki/Sampling_\(signal_processing\)](https://en.wikipedia.org/wiki/Sampling_(signal_processing))
4. ⁴Image source: Gilda Raoof Mehrpoor et al. (2016). "Sampling of a signal in the frequency and time domain."
<https://www.researchgate.net/publication/308748652>

✓ Sampling and Reconstruction

Exercise 1: Plotting

✓ Sine Wave

```
import numpy as np
import matplotlib.pyplot as plt

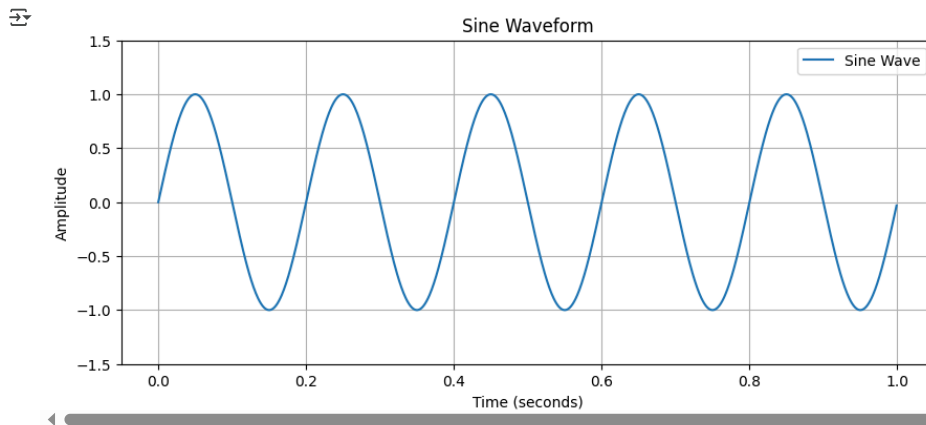
# Parameters
frequency = 5          # Hz
amplitude = 1
phase = 0              # Radians
duration = 1           # Seconds
sample_rate = 1000     # Samples per second

# Time vector
t = np.linspace(0, duration, int(sample_rate * duration), endpoint=False)

# Generate sine wave
y = amplitude * np.sin(2 * np.pi * frequency * t + phase)

# Plot
plt.figure(figsize=(10, 4))
plt.plot(t, y, label='Sine Wave')
plt.title('Sine Waveform')
plt.xlabel('Time (seconds)')
plt.ylabel('Amplitude')
plt.grid(True)
plt.legend()
plt.ylim(-1.5 * amplitude, 1.5 * amplitude)
```

plt.show()



✓ Rectangular wave

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

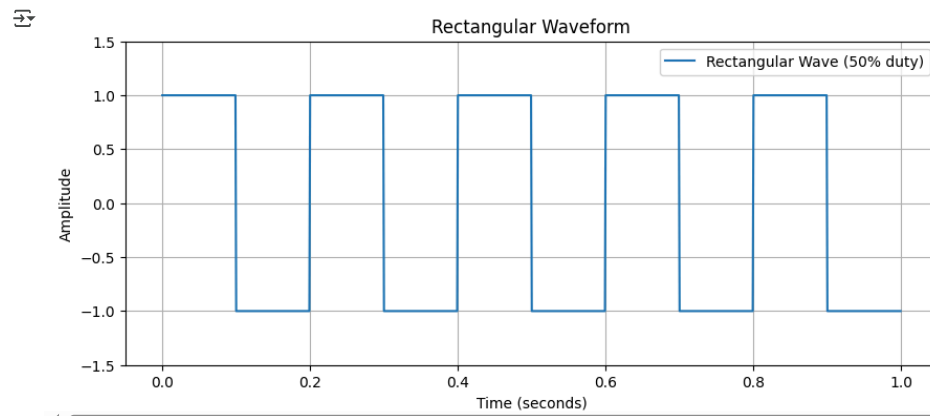
# Parameters
frequency = 5          # Hz
amplitude = 1
duty_cycle = 0.5       # 50% high, 50% low
duration = 1           # seconds
sample_rate = 1000     # Hz

# Time vector
t = np.linspace(0, duration, int(sample_rate * duration), endpoint=False)

# Generate rectangular wave
rect_wave = amplitude * signal.square(2 * np.pi * frequency * t, duty=duty_cycle)

# Plot
plt.figure(figsize=(10, 4))
plt.plot(t, rect_wave, label=f'Rectangular Wave ({int(duty_cycle * 100)}% duty)')
plt.title('Rectangular Waveform')
plt.xlabel('Time (seconds)')
```

```
plt.ylabel('Amplitude')
plt.ylim(-1.5 * amplitude, 1.5 * amplitude)
plt.grid(True)
plt.legend()
plt.show()
```



▼ Triangular wave

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

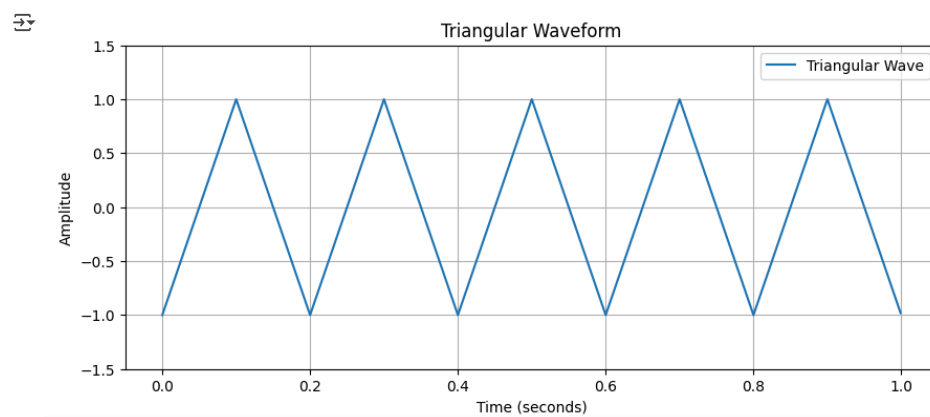
# Parameters
frequency = 5          # Hz
amplitude = 1
duration = 1           # seconds
sample_rate = 1000     # Hz

# Time vector
t = np.linspace(0, duration, int(sample_rate * duration), endpoint=False)

# Generate triangular wave (symmetric sawtooth with width=0.5)
tri_wave = amplitude * signal.sawtooth(2 * np.pi * frequency * t, width=0.5)

# Plot
plt.figure(figsize=(10, 4))
```

```
plt.plot(t, tri_wave, label='Triangular Wave')
plt.title('Triangular Waveform')
plt.xlabel('Time (seconds)')
plt.ylabel('Amplitude')
plt.ylim(-1.5 * amplitude, 1.5 * amplitude)
plt.grid(True)
plt.legend()
plt.show()
```



▼ Exercise 2a: Interactive plot

- Write a function that displays all three signal types in one plot on top of each other in different colors.
- Use sliders to change settings add a button to reset values

```
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display, clear_output
import ipywidgets as widgets

# Time vector
t = np.linspace(0, 1, 1000)

# Default values
default_freq = 5
default_amp = 1
```

```

# Widgets: Frequency, Amplitude, and Reset Button
freq_slider = widgets.FloatSlider(value=default_freq, min=1, max=20, step=0.5, description='Frequency (Hz):')
amp_slider = widgets.FloatSlider(value=default_amp, min=0.1, max=5, step=0.1, description='Amplitude:')
reset_button = widgets.Button(description='Reset', button_style='warning')

# Plot function
def plot_signals(freq, amp):
    sine = amp * np.sin(2 * np.pi * freq * t)
    square = amp * np.sign(np.sin(2 * np.pi * freq * t))
    triangle = amp * (2 * np.abs(2 * (t * freq % 1) - 1) - 1)

    plt.figure(figsize=(10, 5))
    plt.plot(t, sine, label='Sine Wave', color='blue')
    plt.plot(t, triangle, label='Triangle Wave', color='green')
    plt.plot(t, square, label='Square Wave', color='red')

    plt.title('Sine, Triangle, and Square Waves')
    plt.xlabel('Time [s]')
    plt.ylabel('Amplitude')
    plt.grid(True)
    plt.legend()
    plt.ylim(-amp - 0.5, amp + 0.5)
    plt.tight_layout()
    plt.show()

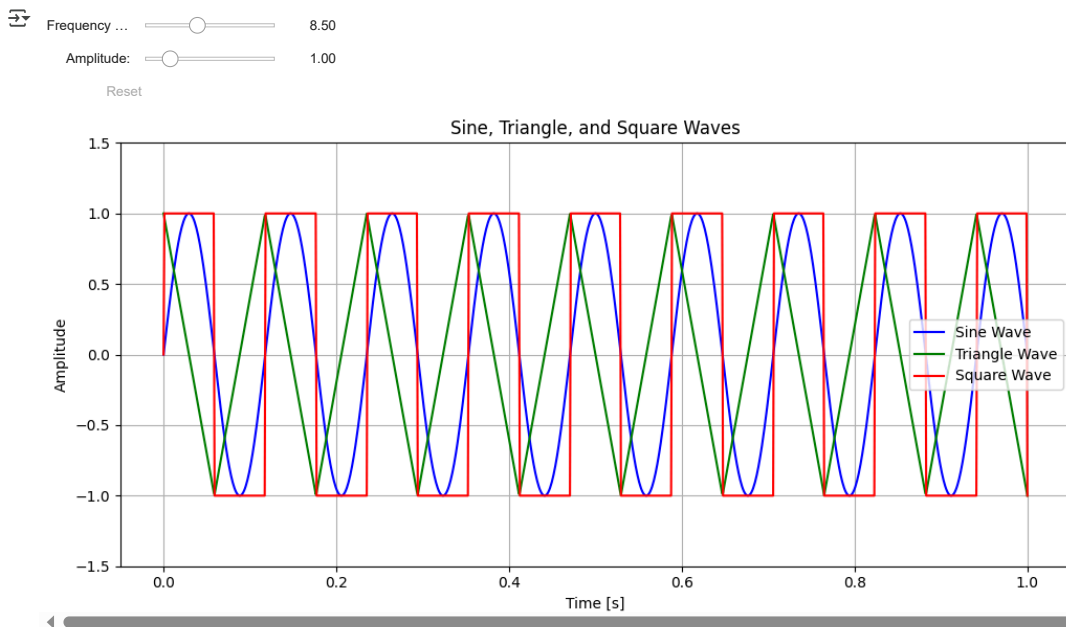
# Update function
def update_plot(change=None):
    clear_output(wait=True)
    display(ui)
    plot_signals(freq_slider.value, amp_slider.value)

# Reset function
def reset_values(b):
    freq_slider.value = default_freq
    amp_slider.value = default_amp
    update_plot()

# Bind events
freq_slider.observe(update_plot, names='value')
amp_slider.observe(update_plot, names='value')
reset_button.on_click(reset_values)

# Display UI
ui = widgets.VBox([freq_slider, amp_slider, reset_button])
display(ui)
plot_signals(default_freq, default_amp)

```



Exercise 2b: Interactive addition of two sine signals

Write a function that adds and displays two sinusoidal signals.

- The signals should be adjustable in frequency and phase
- The individual signals and the resulting signal should be displayed

```

import numpy as np
import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display, clear_output

# --- Plotting Function ---
def plot_sum_of_sines(freq1, phase1, freq2, phase2, duration=1):
    t = np.linspace(0, duration, 1000)

```

```

t = np.linspace(0, duration, 1000)

# Two individual sine waves
y1 = np.sin(2 * np.pi * freq1 * t + np.deg2rad(phase1))
y2 = np.sin(2 * np.pi * freq2 * t + np.deg2rad(phase2))

# Sum of the two sine waves
y_sum = y1 + y2

# Plotting
plt.figure(figsize=(10, 5))
plt.plot(t, y1, label=f'Signal 1: {freq1} Hz, {phase1}°', color='blue')
plt.plot(t, y2, label=f'Signal 2: {freq2} Hz, {phase2}°', color='green')
plt.plot(t, y_sum, label='Sum', color='red', linewidth=2)
plt.xlabel('Time (seconds)')
plt.ylabel('Amplitude')
plt.title('Sum of Two Sine Waves')
plt.ylim(-2.5, 2.5)
plt.grid(True)
plt.legend()
plt.show()

# --- Widgets ---
freq1_slider = widgets.FloatSlider(value=2, min=0.1, max=10, step=0.1, description='Freq 1:')
phase1_slider = widgets.IntSlider(value=0, min=0, max=360, step=1, description='Phase 1 (°):')

freq2_slider = widgets.FloatSlider(value=3, min=0.1, max=10, step=0.1, description='Freq 2:')
phase2_slider = widgets.IntSlider(value=0, min=0, max=360, step=1, description='Phase 2 (°):')

output = widgets.Output()

# --- Update Function ---
def update_plot(change=None):
    with output:
        clear_output(wait=True)
        plot_sum_of_sines(
            freq1=freq1_slider.value,
            phase1=phase1_slider.value,
            freq2=freq2_slider.value,
            phase2=phase2_slider.value
        )

# --- Bind Sliders to Update Function ---
for slider in [freq1_slider, phase1_slider, freq2_slider, phase2_slider]:
    slider.observe(update_plot, names='value')

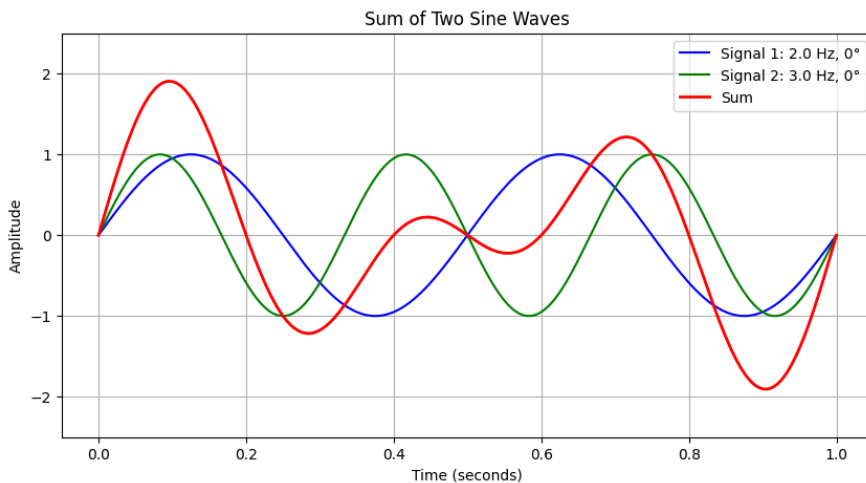
# --- Display UI ---
display(widgets.VBox([
    widgets.HBox([freq1_slider, phase1_slider]),
    widgets.HBox([freq2_slider, phase2_slider]),
    output
]))
'''

# --- Initial Plot ---
update_plot()

```



Freq 1: Phase 1 (°):
 Freq 2: Phase 2 (°):



Exercise 3a: Sampling

Try out sampling of the different signals

- Sine signal
- Triangular signal
- Square wave signal

```

import numpy as np
import matplotlib.pyplot as plt

```

```

# Signal generators
def generate_sine_wave(amplitude, frequency, phase, duration):

```

```

t = np.linspace(0, duration, 10000)
y = amplitude * np.sin(2 * np.pi * frequency * t + phase)
return t, y

def generate_square_wave(amplitude, frequency, duty_cycle, phase, duration):
    t = np.linspace(0, duration, 10000)
    y = amplitude * np.sign(np.sin(2 * np.pi * frequency * t))
    return t, y

def generate_triangle_wave(amplitude, frequency, phase, duration):
    t = np.linspace(0, duration, 10000)
    y = 2 * amplitude * np.abs(2 * np.mod(frequency * t + phase, 1) - 1) - amplitude
    return t, y

# Sampling function
def sample_waveform(original_t, waveform, sampling_frequency, duration):
    sample_t = np.linspace(0, duration, int(duration * sampling_frequency))
    sampled_y = np.interp(sample_t, original_t, waveform)
    return sample_t, sampled_y

# Plotting function
def plot_sampling_comparison(signal_func, name, amplitude=1, frequency=10, phase=0, duty=0.5, duration=1.0):
    original_t, y = signal_func(amplitude, frequency, phase, duration) if name != "Square" else signal_func(amplitude, frequency, duty, phase, duration)

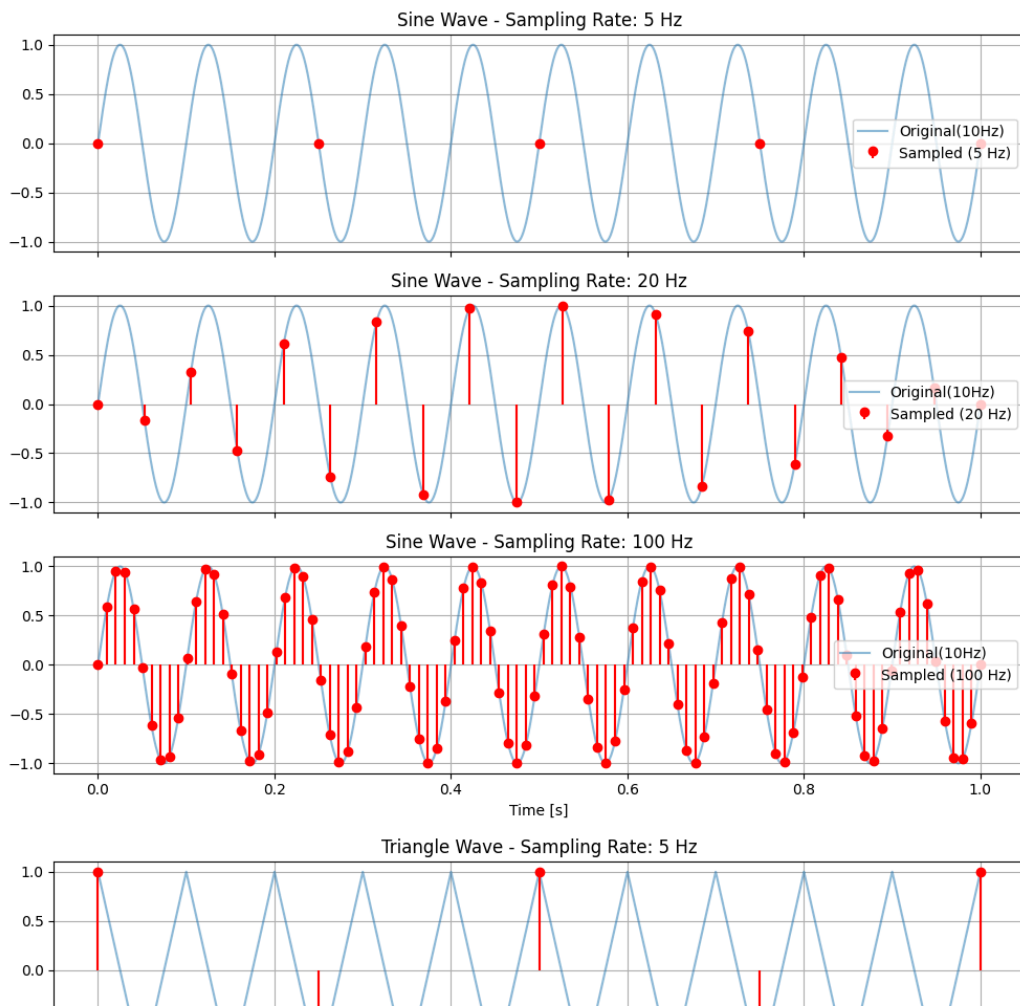
    fig, axs = plt.subplots(3, 1, figsize=(10, 8), sharex=True)
    sampling_rates = [5, 20, 100] # Hz

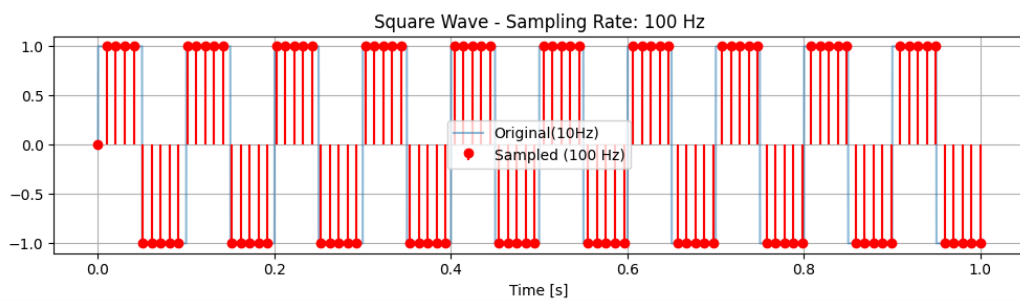
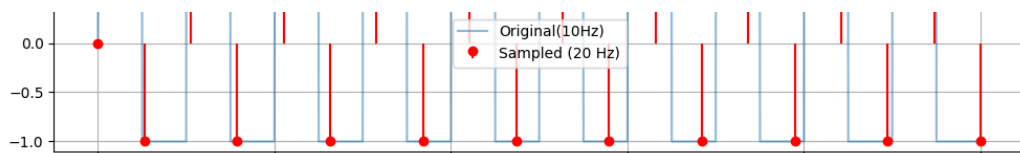
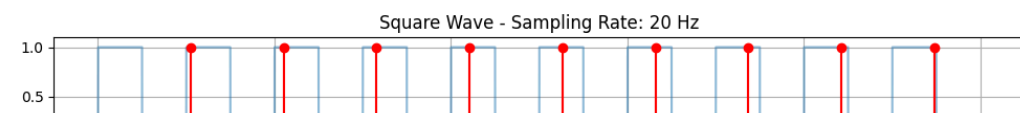
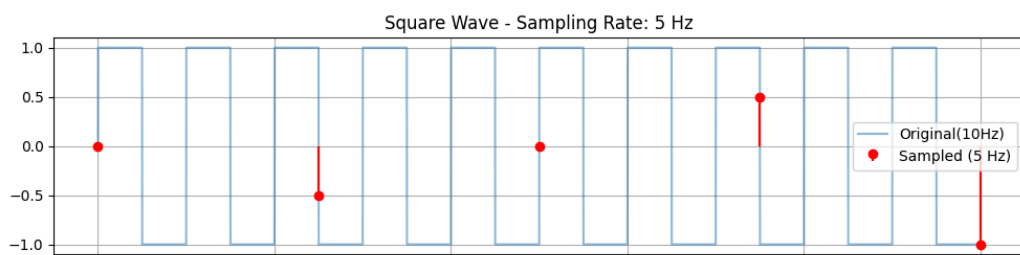
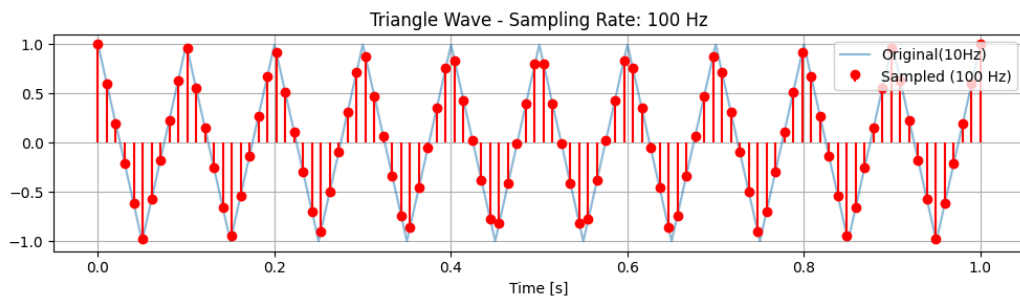
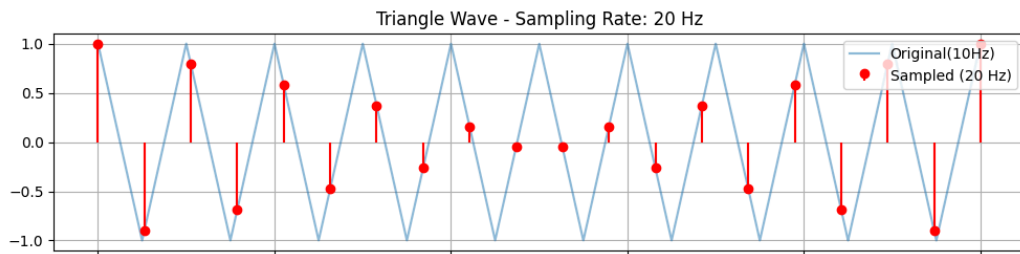
    for i, fs in enumerate(sampling_rates):
        sample_t, sample_y = sample_waveform(original_t, y, fs, duration)
        axs[i].plot(original_t, y, label="Original(10Hz)", alpha=0.5)
        axs[i].stem(sample_t, sample_y, linefmt='r-', markerfmt='ro', basefmt=' ', label=f"Sampled ({fs} Hz)")
        axs[i].set_title(f"{name} Wave - Sampling Rate: {fs} Hz")
        axs[i].legend()
        axs[i].grid(True)

    axs[-1].set_xlabel("Time [s]")
    plt.tight_layout()
    plt.show()

# Try it out for all three signals:
plot_sampling_comparison(generate_sine_wave, "Sine")
plot_sampling_comparison(generate_triangle_wave, "Triangle")
plot_sampling_comparison(generate_square_wave, "Square")

```





1. Question

What observations can be made from sampling at different frequencies?

The signal frequency is **10 Hz**.

- At a **low sampling frequency** (e.g., **5 Hz**), The sampled signal does not resemble the original waveform and fails to capture it, information about the waves is lost and it is misrepresented. By looking at the sampled signals, it is evident that reconstruction will be difficult due to the loss of information. It is also called undersampling according to the Sampling theorem and may lead to aliasing
- At **twice the signal frequency** (e.g., **20 Hz**) The sampled wave begins to appear as original for sine wave, but it is still not very smooth. The sampled wave does not appears as original for triangular and square waves.
- At a **high sampling frequency** (e.g., **100 Hz**), At a high sampling frequency (100 Hz), the sampled signal closely matches the original waveform.

2. Question

What frequency is necessary to recognise the signal?

For original signal of $f = 10\text{Hz}$ to recognise the signals it is necessary that

- sine wave: sampling frequency $f_s \geq 2f$ i.e., 20 Hz
- Triangle wave: $f_s \geq 5f$ i.e., 40-50 Hz
- Square wave: $f_s \geq 10f$ i.e., 80-100 Hz

✓ Exercise 3b: Advanced sampling methods

Create a plot using different sampling functions:

- Zero-order hold
- Sinc Reconstruction
- Linear interpolation of sampled values

```
import numpy as np
import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display, clear_output

# ---- Signal Generators ----
def generate_signals(frequency, duration, sample_rate=44100):
    t = np.linspace(0, duration, int(duration * sample_rate))
    sine = np.sin(2 * np.pi * frequency * t)
    triangle = 2 * np.abs(2 * (t * frequency % 1) - 1) - 1

    square = np.sign(np.sin(2 * np.pi * frequency * t))
    return t, sine, triangle, square

# ---- Sampling Functions ----
def sample_signal(signal_func, sample_times, frequency):
    return signal_func(2 * np.pi * frequency * sample_times)

# ---- ZOH Reconstruction ----
def zero_order_hold(sampled_signal, sample_times, t):
    reconstructed = np.zeros_like(t)
    sample_indices = np.searchsorted(t, sample_times, side='right') - 1
    sample_indices = np.clip(sample_indices, 0, len(t) - 1)
    for i in range(len(sample_times) - 1):
        reconstructed[sample_indices[i]:sample_indices[i + 1]] = sampled_signal[i]
    reconstructed[sample_indices[-1]:] = sampled_signal[-1]
    return reconstructed

# ---- Sinc Reconstruction ----
def sinc_reconstruct(sampled_signal, sample_times, t, sample_rate):
    reconstructed = np.zeros_like(t)
    for i, ts in enumerate(sample_times):
        reconstructed += sampled_signal[i] * np.sinc((t - ts) * sample_rate)
    return reconstructed

# ---- Reconstruction Plot Function ----
def plot_reconstruction(signal_type, sampling_rate):
    duration = 1
    frequency = 10
    high_sample_rate = 44100 # For "continuous" signal
    t, sine, triangle, square = generate_signals(frequency, duration, high_sample_rate)

    # Get the original & sample signals
    sample_times = np.linspace(0, duration, int(duration * sampling_rate))
    if signal_type == 'Sine':
        original = sine
        sampled = np.sin(2 * np.pi * frequency * sample_times)
        signal_func = np.sin
    elif signal_type == 'Triangle':
        original = triangle
        sampled = 2 * np.abs(2 * (sample_times * frequency % 1) - 1) - 1
        signal_func = lambda x: 2 * np.abs(2 * ((x / (2 * np.pi)) % 1) - 1) - 1
    elif signal_type == 'Square':
        original = square
        sampled = np.sign(np.sin(2 * np.pi * frequency * sample_times))
        signal_func = lambda x: np.sign(np.sin(x))

    # Reconstruct
    linear_interp = np.interp(t, sample_times, sampled)
    zoh = zero_order_hold(sampled, sample_times, t)
    sinc_interp = sinc_reconstruct(sampled, sample_times, t, sampling_rate)

    # Plotting
```



```

# Plotting
for method, y_rec, color in zip(
    ['linear Interpolation', 'Zero-Order Hold', 'Sinc Reconstruction'],
    [linear_interp, zoh, sinc_interp],
    ['green', 'blue', 'magenta']
):
    plt.figure(figsize=(10, 4))
    plt.plot(t, original, 'k--', label='Original Signal (10Hz)', linewidth=1)
    plt.plot(sample_times, sampled, 'ro', label='Sampled Points')
    plt.plot(t, y_rec, color=color, label=method)
    plt.title(f'{signal_type} Wave - {method} (Sampling Rate = {sampling_rate} Hz)')
    plt.xlabel('Time [s]')
    plt.ylabel('Amplitude')
    plt.ylim(-1.5, 1.5)
    plt.grid(True)
    plt.legend()
    plt.show()

# ---- Interactive Controls ----
signal_selector = widgets Dropdown(options=['Sine', 'Triangle', 'Square'], value='Sine', description='Signal:')
sampling_slider = widgets IntSlider(value=10, min=1, max=100, step=1, description='Samp_rate:')
output = widgets.Output()

def update_plot(change):
    with output:
        clear_output(wait=True)
        plot_reconstruction(signal_selector.value, sampling_slider.value)

signal_selector.observe(update_plot, names='value')
sampling_slider.observe(update_plot, names='value')

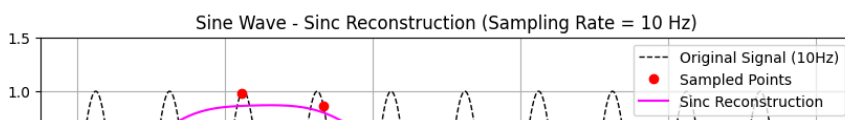
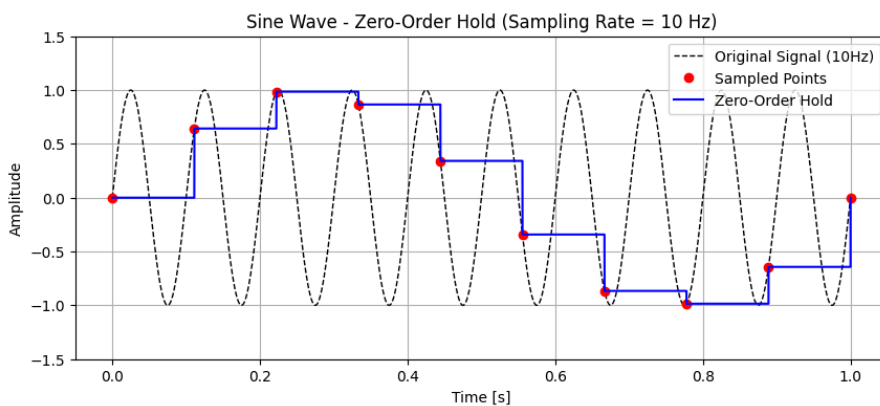
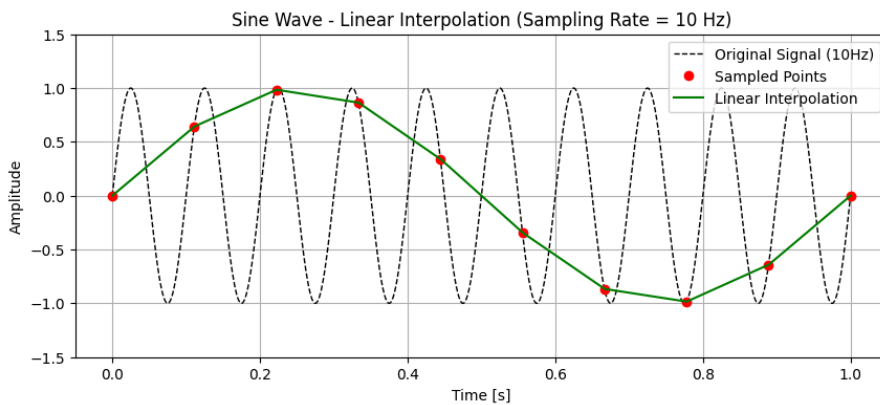
display(widgets.VBox([signal_selector, sampling_slider]), output)
update_plot(None)

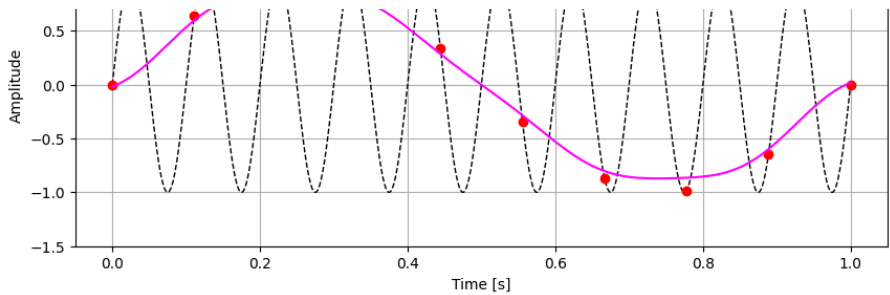
```



Signal:

Samp_rate:





1. Question

What differences can be seen?

- Linear Interpolation

Linear interpolation is done by connecting the sampled points with straight lines, making the shape closer to the original but still not smooth. This method suits best for sine and triangle waves, but not for square waves. The edges of the square wave are not completely recovered after reconstruction.

- Zero-Order Hold (ZOH)

This method suits best only for square waves. It creates a staircase effect for sine and triangle waves even at higher frequency; it is not best suited for both the waves.

- Sinc Reconstruction

This method is best for sine and triangle waves. It is not the best for square waves as it produces oscillations at the peaks of the wave.

```
from IPython.display import display, Markdown

# Creating the result/observation table in markdown format
table = """
| **Signal Type** | **Method** | **Low Frequency(8 Hz)** | **Frequency (20 Hz)** | |
|---|---|---|---|---|
| **Sine Wave** | **Linear Interpolation** | Appears like a triangular wave at lower frequency | Appears like a triangular wave with sharp edges; almost same as original frequency |
| | **Zero-Order Hold (ZOH)** | Appears like a square wave at lower frequency | Appears like a square wave with staircase edges; almost same as original frequency |
| | **Sinc Reconstruction** | Appears like a sine wave at lower frequency; Amplitude of the sampled signal is exceeded/overshooting the range of the original signal | Appears like a sine wave at same frequency; Minor overshoot |
| **Triangle Wave** | **Linear Interpolation** | Appears like a triangular wave at lower frequency | Appears like a triangular wave; same as original frequency; Amplitude of the wave is not exceeded |
| | **Zero-Order Hold (ZOH)** | Appears like a stepped wave at lower frequency | Appears like a stepped wave; same as original frequency; Amplitude of the wave is not exceeded |
| | **Sinc Reconstruction** | Appears like a sine wave at lower frequency; Minor overshoot | Appears like a sine wave almost at same frequency; Minor overshoot; Amplitude of the wave is exceeded |
| **Square Wave** | **Linear Interpolation** | Appears like a triangular wave at lower frequency | Appears like a triangular wave at same frequency and amplitude | Appears like a triangular wave at same frequency and amplitude |
| | **Zero-Order Hold (ZOH)** | Appears like a square wave at lower frequency | Appears like square wave; same as original frequency; not exactly following the original signal |
| | **Sinc Reconstruction** | Appears like a sine wave at lower frequency; Minor overshoot | Appears like a sine wave at same frequency; Minor overshoot | Appears like a sine wave at same frequency; Minor overshoot |

# Displaying the table as markdown
display(Markdown(table))
```